Lucrare de laborator 1
Manipularea imaginilor cu OpenCV

Sarcini

- Pregătiți un set de **8–12 imagini** de test: imaginile din arhiva cursului (`opencv_bootcamp_assets_NB1.zip`), imaginile de test incluse în `scikit-image` și **cel puțin 3 imagini proprii** (un peisaj, un document/text fotografiat, un obiect cu o culoare distinctă, o scenă slab iluminată).
- Încărcați imaginile, verificați **atributele** (dimensiuni, canale, tip de date, interval de valori, histogramă) și afișați-le corect (BGR → RGB).
- Realizați și **evaluați cantitativ** următoarele operații:
  1. **Transformări geometrice** (decupare, flip, rotire, redimensionare): comparați metodele de interpolare după timpul de execuție și calitatea reconstrucției (PSNR, SSIM), separat pentru micșorare și mărire.
  2. **Luminozitate, contrast, gamma**: tratați corect overflow-ul; măsurați media, contrastul (deviația standard) și procentul de pixeli saturați.
  3. **Spații de culori** (Gray, HSV, LAB, YCrCb): separați/îmbinați canale; segmentați un obiect după culoare în RGB și în HSV și măsurați stabilitatea (IoU) la schimbări de iluminare.
  4. **Praguri și măști**: comparați pragul global, Otsu și pragul adaptiv (scor F1 pe pixelii de text); combinați două imagini prin măști și operații pe biți.
  5. **Adnotare** (linii, dreptunghiuri, cercuri, text) și un **colaj** care folosește cel puțin 3 concepte de mai sus.
  6. **Video**: aplicați pipeline-ul *redimensionare* → *segmentare pe culoare* → *adnotare* cadru cu cadru pe un clip scurt, salvați rezultatul și măsurați timpul de procesare per cadru (FPS).
- Agregați rezultatele în tabele și grafice și formulați concluzii **bazate pe valorile măsurate**, nu doar pe impresii vizuale.

Livrabil: notebook-ul completat și rulat, împreună cu exportul PDF (`LL1_CV_NumePrenume.ipynb` / `.pdf`) și directorul `rezultate/` (fișiere JSON, colajul, clipul adnotat).

Completați variabilele de mai jos conform setării voastre:

- `IMG_DIR` : directorul cu imaginile proprii (implicit `imagini/`).
- `VIDEO_PATH` : clipul video de test (implicit `roboti.mp4` , disponibil pe pagina cursului; puteți folosi orice clip scurt propriu).
- `OUT_DIR` : directorul în care se salvează rezultatele (implicit `rezultate/`).

Instrucțiuni de instalare

1. Python 3.10+ (recomandat într-un mediu virtual: `python -m venv .venv`).

2. Instalați pachetele: `pip install opencv-contrib-python numpy pandas matplotlib seaborn scikit-image ipywidgets jupyterlab`
3. Verificați instalarea: `python -c "import cv2; print(cv2.__version__)"` (așteptat ≥ 4.8).
4. Resurse: arhiva `opencv_bootcamp_assets_NB1.zip` se descarcă automat din notebook (link); imaginile de test `scikit-image` sunt incluse în pachet; clipul `roboti.mp4` se descarcă de pe pagina cursului.

Google Colab: pachetele sunt preinstalate; încărcați în sesiune `roboti.mp4` și imaginile proprii. `cv2.imshow()` nu funcționează în notebook (nici local, nici în Colab) — afișați imaginile cu `matplotlib`, prin funcția `show()` definită mai jos.

Exemplu de realizare

Pregătirea spațiului de lucru

In [1]:
```
import os, time, json
from glob import glob
from pathlib import Path
from zipfile import ZipFile
from urllib.request import urlretrieve

import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import ipywidgets as widgets
from ipywidgets import interact
from skimage import data as skdata
from skimage.metrics import peak_signal_noise_ratio, structural_similarity

%matplotlib inline
sns.set_theme(style='whitegrid')
print('OpenCV', cv2.__version__, '| NumPy', np.__version__)
```

```
OpenCV 4.13.0 | NumPy 2.4.4
```

In [2]:
```
IMG_DIR    = 'imagini'
VIDEO_PATH = 'roboti.mp4'
OUT_DIR    = 'rezultate'

os.makedirs(OUT_DIR, exist_ok=True)
os.makedirs(IMG_DIR, exist_ok=True)
```

Funcția `download_and_unzip(...)` (aceeași ca la curs) descarcă și dezarhivează resursele. Arhiva se descarcă o singură dată.

In [3]:
```
def download_and_unzip(url, save_path):
    print('Se descarcă și se dezarhivează resursele...', end='')
    urlretrieve(url, save_path)
    with ZipFile(save_path) as z:
        z.extractall(os.path.split(save_path)[0])
    print(' gata.')


URL = 'https://www.dropbox.com/s/qhhlqcica1nvtaw/opencv_bootcamp_assets_NB1.zip?dl=1'
asset_zip_path = os.path.join(os.getcwd(), 'opencv_bootcamp_assets_NB1.zip')
```

```
if not os.path.exists(asset_zip_path):
    download_and_unzip(URL, asset_zip_path)
```

OpenCV lucrează cu imagini **BGR**, iar `matplotlib` afișează **RGB**. Funcția `show()` afișează o listă de imagini (color sau gri) într-o grilă, făcând conversia corectă; o vom folosi pe tot parcursul lucrării. `timeit_ms()` măsoară timpul mediu de execuție al unei funcții, iar `save_json()` exportă un `DataFrame` în directorul de rezultate.

In [4]:
```
def show(images, titles=None, cols=None, size=4, cmap='gray'):
    n = len(images)
    cols = cols or n
    rows = int(np.ceil(n / cols))
    plt.figure(figsize=(size * cols, size * rows * 0.8))

    for i, img in enumerate(images):
        plt.subplot(rows, cols, i + 1)
        if img.ndim == 2:
            plt.imshow(img, cmap=cmap, vmin=0, vmax=255)
        else:
            plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        if titles:
            plt.title(titles[i])
        plt.axis('off')

    plt.tight_layout()
    plt.show()


def timeit_ms(fn, repeats=10):
    start = time.perf_counter()
    for _ in range(repeats):
        fn()
    return (time.perf_counter() - start) / repeats * 1000


def save_json(df, name):
    path = os.path.join(OUT_DIR, name)
    df.to_json(path, orient='records', indent=4, index=False)
```

Setul de imagini

Folosim trei surse: arhiva cursului (`cv2.imread`), imaginile de test din `scikit-image` (care vin în format **RGB** și trebuie convertite în BGR, convenția OpenCV) și imaginile proprii din directorul `IMG_DIR` .

In [5]:
```
def from_skimage(name):
    rgb = getattr(skdata, name)()
    if rgb.ndim == 2:
        return rgb                          # imagine în tonuri de gri
    return cv2.cvtColor(rgb[..., :3], cv2.COLOR_RGB2BGR)


IMAGES = {
    'lac':       cv2.imread('New_Zealand_Lake.jpg'),
    'logo':      cv2.imread('coca-cola-logo.png'),
    'sah_color': cv2.imread('checkerboard_color.png'),
    'astronaut': from_skimage('astronaut'),
    'cafea':     from_skimage('coffee'),
    'racheta':   from_skimage('rocket'),
    'pisica':    from_skimage('chelsea'),
    'hubble':    from_skimage('hubble_deep_field'),
    'pagina':    from_skimage('page'),
}
```

```
for path in sorted(glob(os.path.join(IMG_DIR, '*'))):
    img = cv2.imread(path)
    if img is not None:
        IMAGES[Path(path).stem] = img

print(len(IMAGES), 'imagini:', ', '.join(IMAGES))
```

```
9 imagini: lac, logo, sah_color, astronaut, cafea, racheta, pisica, hubble, pagina
```

Verificăm atributele fiecărei imagini: forma `(H, W, C)`, tipul de date (`uint8` → valori în `[0, 255]`), intervalul de valori și memoria ocupată.

In [6]:
```
rows = []

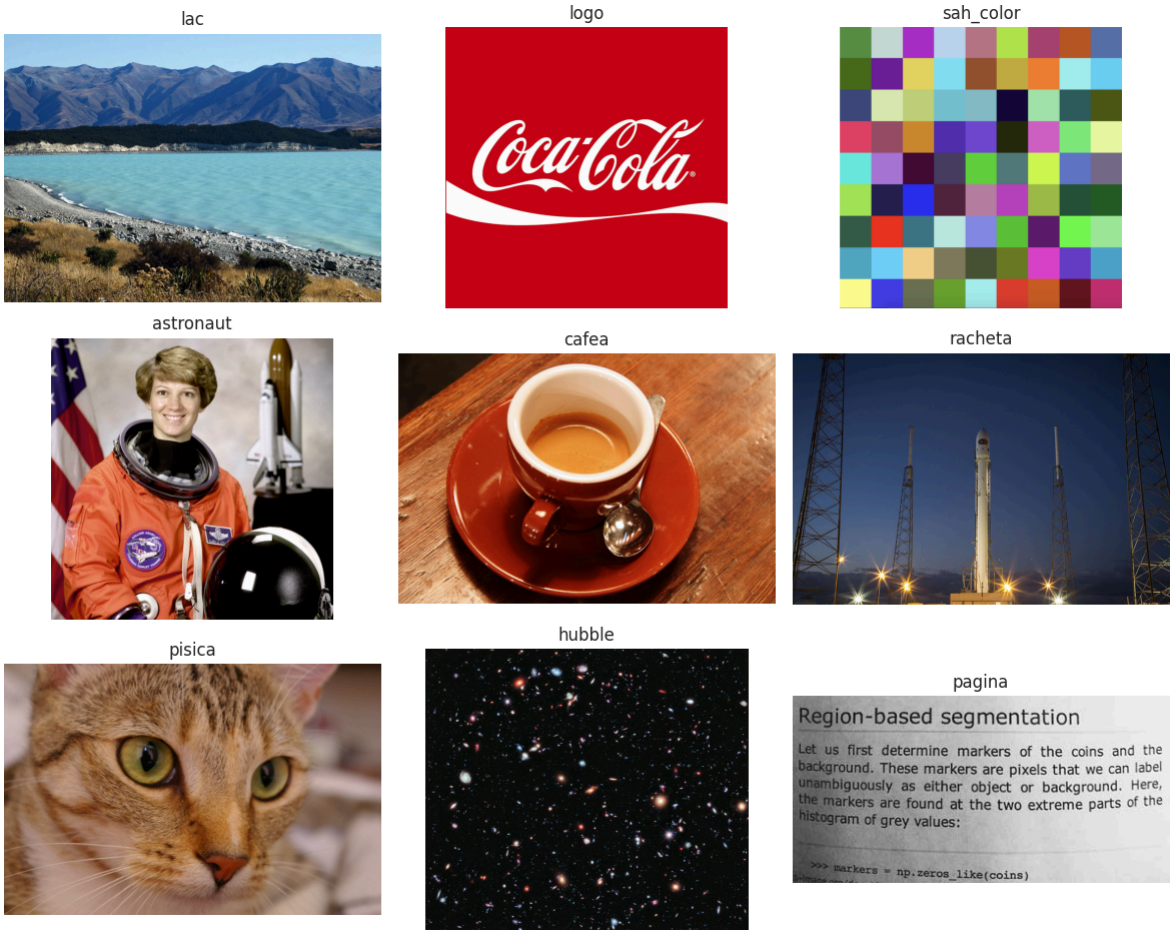
for name, img in IMAGES.items():
    rows.append({
        'imagine': name,
        'inaltime': img.shape[0],
        'latime': img.shape[1],
        'canale': 1 if img.ndim == 2 else img.shape[2],
        'dtype': str(img.dtype),
        'min': int(img.min()),
        'max': int(img.max()),
        'medie': round(float(img.mean()), 1),
        'MB': round(img.nbytes / 2**20, 2),
    })

attrs_df = pd.DataFrame(rows)
attrs_df
```

Out[6]:

	imagine	inaltime	latime	canale	dtype	min	max	medie	MB
0	lac	600	840	3	uint8	0	255	122.6	1.44
1	logo	700	700	3	uint8	0	255	94.4	1.40
2	sah_color	1800	1800	3	uint8	0	255	130.4	9.27
3	astronaut	512	512	3	uint8	0	255	114.6	0.75
4	cafea	400	600	3	uint8	0	255	98.6	0.69
5	racheta	427	640	3	uint8	0	255	65.3	0.78
6	pisica	300	451	3	uint8	0	231	115.3	0.39
7	hubble	872	1000	3	uint8	0	255	19.2	2.49
8	pagina	191	384	1	uint8	0	255	171.5	0.07

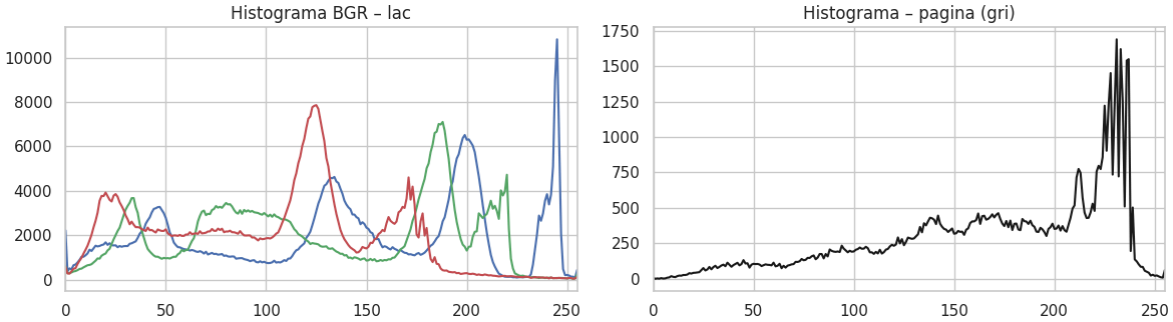
In [7]:
```
show(list(IMAGES.values()), list(IMAGES.keys()), cols=3, size=4)
```



Histograma arată distribuția intensităților pe fiecare canal (`cv2.calcHist`). Pentru imaginea `lac` dominanța canalului albastru este vizibilă; pentru `pagina` (document fotografiat, iluminare neuniformă) histograma este bimodală — text (întunecat) și hârtie (luminoasă) — dar cu suprapunere.

In [8]:
```
def plot_hist(img, ax, title):
    channels = [(0, 'k')] if img.ndim == 2 else [(0, 'b'), (1, 'g'), (2, 'r')]
    for c, color in channels:
        ax.plot(cv2.calcHist([img], [c], None, [256], [0, 256]), color=color)
    ax.set_title(title)
    ax.set_xlim(0, 255)


fig, axes = plt.subplots(1, 2, figsize=(12, 3.5))
plot_hist(IMAGES['lac'], axes[0], 'Histograma BGR – lac')
plot_hist(IMAGES['pagina'], axes[1], 'Histograma – pagina (gri)')
plt.tight_layout()
plt.show()
```



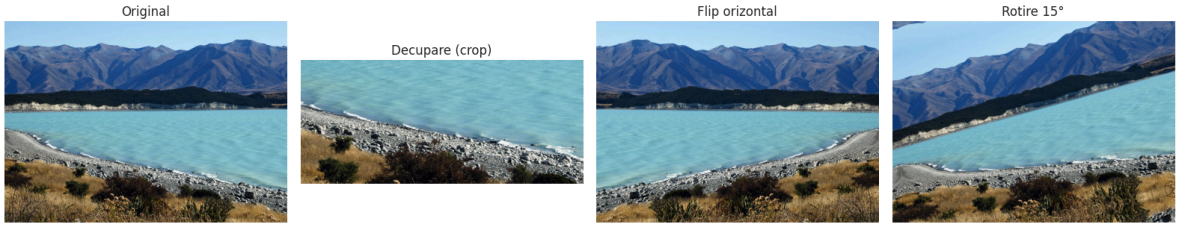
1. Transformări geometrice

Decuparea este o simplă indexare NumPy `img[y1:y2, x1:x2]`; `cv2.flip` inversează pe orizontală (`1`), verticală (`0`) sau ambele (`-1`); rotirea în jurul centrului se face cu o matrice afină (`cv2.getRotationMatrix2D`) aplicată prin `cv2.warpAffine`.

In [9]:
```
img = IMAGES['lac']
h, w = img.shape[:2]

crop = img[300:520, 150:650]
flip_h = cv2.flip(img, 1)
M = cv2.getRotationMatrix2D(center=(w / 2, h / 2), angle=15, scale=1.0)
rot = cv2.warpAffine(img, M, (w, h), borderMode=cv2.BORDER_REFLECT)

show([img, crop, flip_h, rot],
     ['Original', 'Decupare (crop)', 'Flip orizontal', 'Rotire 15°'], cols=4)
```



Comparația metodelor de interpolare

`cv2.resize` acceptă mai multe metode de interpolare, iar alegerea potrivită depinde de direcția redimensionării. Le comparăm **obiectiv**, pe două roluri:

- **micșorare**: imaginea este redusă la 25% cu metoda testată, apoi mărită înapoi cu o metodă fixă de calitate (`LANCZOS4`);
- **mărire**: imaginea este redusă la 25% cu metoda fixă recomandată pentru micșorare (`AREA`), apoi mărită înapoi cu metoda testată.

Pentru fiecare caz măsurăm timpul pasului testat și calitatea reconstrucției față de original: **PSNR** (dB, mai mare = mai fidel) și **SSIM** (în `[0, 1]`, mai aproape de 1 = structură mai bine păstrată).

In [10]:
```
INTERP = {
    'NEAREST':  cv2.INTER_NEAREST,
    'LINEAR':   cv2.INTER_LINEAR,
    'AREA':     cv2.INTER_AREA,
    'CUBIC':    cv2.INTER_CUBIC,
    'LANCZOS4': cv2.INTER_LANCZOS4,
}
SCALE = 0.25


def shrink(img, interp):
    return cv2.resize(img, None, fx=SCALE, fy=SCALE, interpolation=interp)


def enlarge(small, img, interp):
    return cv2.resize(small, (img.shape[1], img.shape[0]), interpolation=interp)
```

```
def quality(img, rec):
    return {
        'psnr': peak_signal_noise_ratio(img, rec, data_range=255),
        'ssim': structural_similarity(img, rec, channel_axis=2, data_range=255),
    }


rows = []

for name in ['lac', 'astronaut', 'cafea', 'racheta', 'pisica']:
    img = IMAGES[name]
    small_area = shrink(img, cv2.INTER_AREA)

    for label, interp in INTERP.items():
        # rolul „micșorare”: metoda testată la micșorare, LANCZOS4 la mărire
        ms = timeit_ms(lambda: shrink(img, interp))
        rec = enlarge(shrink(img, interp), img, cv2.INTER_LANCZOS4)
        rows.append({'imagine': name, 'metoda': label, 'rol': 'micsorare',
                     'timp_ms': ms, **quality(img, rec)})

        # rolul „mărire”: AREA la micșorare, metoda testată la mărire
        ms = timeit_ms(lambda: enlarge(small_area, img, interp))
        rec = enlarge(small_area, img, interp)
        rows.append({'imagine': name, 'metoda': label, 'rol': 'marire',
                     'timp_ms': ms, **quality(img, rec)})

interp_df = pd.DataFrame(rows)
save_json(interp_df, 'interpolare.json')
interp_df.head()
```

Out[10]:

	imagine	metoda	rol	timp_ms	psnr	ssim
0	lac	NEAREST	micsorare	0.031542	20.321436	0.599691
1	lac	NEAREST	marire	0.210466	23.017544	0.695885
2	lac	LINEAR	micsorare	0.079663	22.556678	0.698059
3	lac	LINEAR	marire	0.222133	23.174641	0.687162
4	lac	AREA	micsorare	0.440586	23.483649	0.711379

In [11]:

```
img = IMAGES['pisica']
small_area = shrink(img, cv2.INTER_AREA)
y, x = 60, 180   # regiune cu detalii fine (blana)

crops = [img[y:y + 100, x:x + 100]]
crops += [enlarge(small_area, img, i)[y:y + 100, x:x + 100] for i in INTERP.values()]

show(crops, ['Original'] + [f'mărire {k}' for k in INTERP], cols=6, size=3)
```



In [12]:

```
interp_agg = interp_df.groupby(['rol', 'metoda']).agg(
    timp_ms=('timp_ms', 'mean'),
    psnr=('psnr', 'mean'),
    ssim=('ssim', 'mean'),
).round(3)

interp_agg.unstack('rol')
```

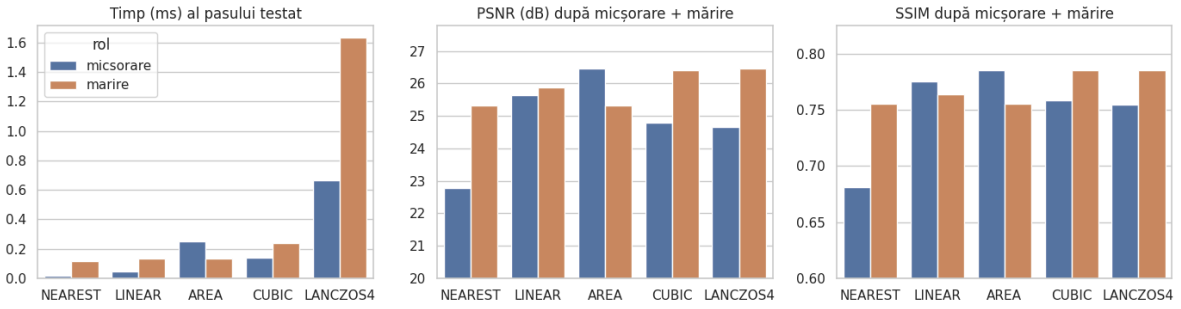
Out[12]:

	timp_ms		psnr		ssim	
rol	marire	micsorare	marire	micsorare	marire	micsorare
metoda						
AREA	0.135	0.252	25.317	26.453	0.755	0.786
CUBIC	0.241	0.141	26.413	24.799	0.785	0.759
LANCZOS4	1.632	0.666	26.453	24.668	0.786	0.755
LINEAR	0.134	0.043	25.885	25.634	0.764	0.775
NEAREST	0.117	0.017	25.329	22.789	0.756	0.681

In [13]:
```
titles = ['Timp (ms) al pasului testat', 'PSNR (dB) după micșorare + mărire',
          'SSIM după micșorare + mărire']
fig, axes = plt.subplots(1, 3, figsize=(14, 3.8))

for ax, metric, title in zip(axes, ['timp_ms', 'psnr', 'ssim'], titles):
    sns.barplot(data=interp_df, x='metoda', y=metric, hue='rol', ax=ax,
                errorbar=None, legend=(metric == 'timp_ms'))
    ax.set_title(title)
    ax.set_xlabel('')
    ax.set_ylabel('')

axes[1].set_ylim(20, None)
axes[2].set_ylim(0.6, None)
plt.tight_layout()
plt.show()
```



2. Luminozitate, contrast și gamma

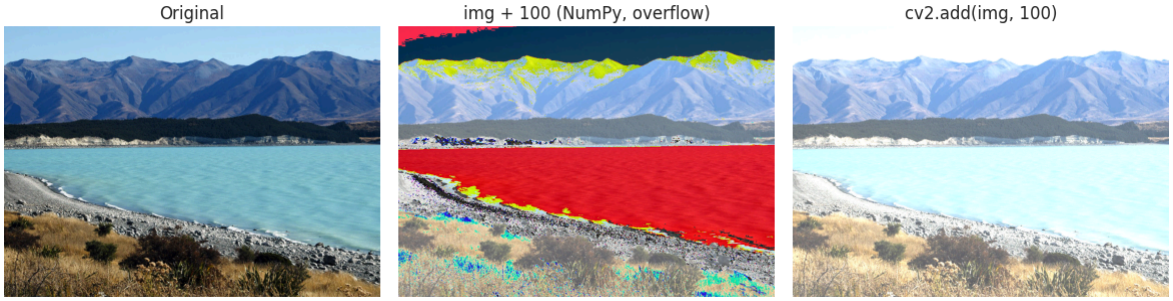
Imaginile sunt `uint8`, deci orice operație aritmetică poate depăși intervalul `[0, 255]`. NumPy face **wrap-around** (`250 + 100 = 94`), în timp ce `cv2.add` / `cv2.subtract` **saturează** (`250 + 100 = 255`). Compararea celor două arată de ce overflow-ul trebuie tratat explicit.

In [14]:
```
a, b = np.full((1, 1), 250, np.uint8), np.full((1, 1), 100, np.uint8)
print('NumPy  250 + 100 =', (a + b)[0, 0], '  (wrap-around)')
print('OpenCV 250 + 100 =', cv2.add(a, b)[0, 0], '  (saturare)')

img = IMAGES['lac']
plus_np = img + 100                                  # overflow: wrap-around
plus_cv = cv2.add(img, 100)                          # saturare la 255
plus_clip = np.clip(img.astype(np.int16) + 100, 0, 255).astype(np.uint8)
print('cv2.add identic cu np.clip:', np.array_equal(plus_cv, plus_clip))

show([img, plus_np, plus_cv],
     ['Original', 'img + 100 (NumPy, overflow)', 'cv2.add(img, 100)'], cols=3)
```

```
NumPy  250 + 100 = 94   (wrap-around)
OpenCV 250 + 100 = 255   (saturare)
cv2.add identic cu np.clip: True
```



Contrastul și luminozitatea se ajustează liniar, `out = α·in + β`, cu `cv2.convertScaleAbs` (înmulțire, adunare și saturare într-un singur pas). Corecția **gamma** este neliniară, `out = 255·(in/255)^γ`, și se aplică eficient printr-un tabel de căutare (`cv2.LUT`): `γ < 1` luminează tonurile întunecate, `γ > 1` le întunecă.

Pentru fiecare variantă măsurăm **media** (luminozitate), **deviația standard** (contrast) și **procentul de pixeli saturați** (0 sau 255), adică informația pierdută ireversibil.

In [15]:
```python
def gamma_correct(img, gamma):
    lut = (255 * (np.arange(256) / 255.0) ** gamma).astype(np.uint8)
    return cv2.LUT(img, lut)


def intensity_stats(img):
    saturated = np.mean((img == 0) | (img == 255))
    return {
        'medie': round(float(img.mean()), 1),
        'std': round(float(img.std()), 1),
        'saturati_%': round(100 * float(saturated), 2),
    }

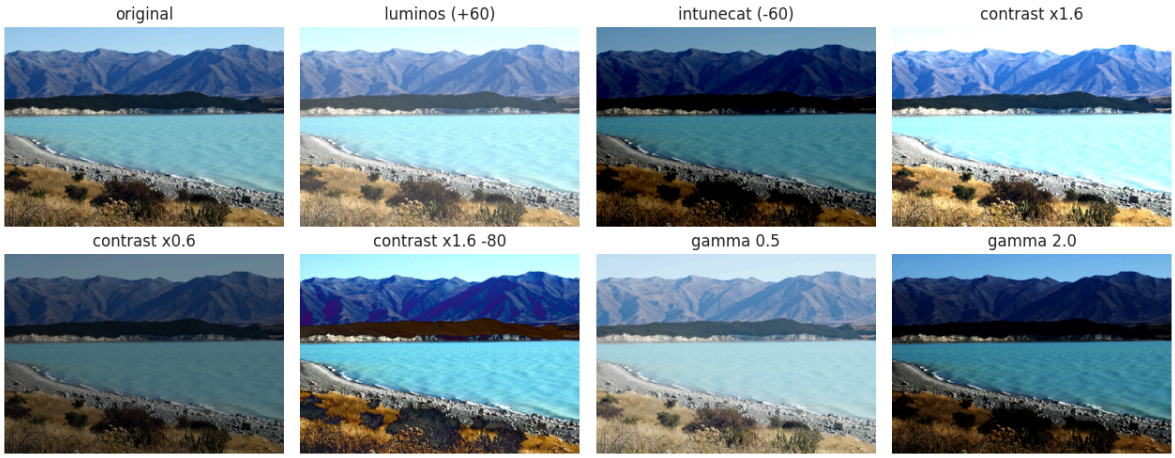

variants = {
    'original':          img,
    'luminos (+60)':     cv2.add(img, 60),
    'intunecat (-60)':   cv2.subtract(img, 60),
    'contrast x1.6':     cv2.convertScaleAbs(img, alpha=1.6, beta=0),
    'contrast x0.6':     cv2.convertScaleAbs(img, alpha=0.6, beta=0),
    'contrast x1.6 -80': cv2.convertScaleAbs(img, alpha=1.6, beta=-80),
    'gamma 0.5':         gamma_correct(img, 0.5),
    'gamma 2.0':         gamma_correct(img, 2.0),
}

bc_df = pd.DataFrame([{'varianta': k, **intensity_stats(v)}
                      for k, v in variants.items()])
save_json(bc_df, 'luminozitate_contrast.json')
bc_df
```

Out[15]:

	varianta	medie	std	saturati_%
0	original	122.6	64.6	0.28
1	luminos (+60)	179.2	59.6	15.76
2	intunecat (-60)	68.4	56.6	21.73
3	contrast x1.6	176.3	79.3	33.41
4	contrast x0.6	73.5	38.8	0.22
5	contrast x1.6 -80	125.7	82.7	8.49
6	gamma 0.5	168.3	52.6	0.28
7	gamma 2.0	74.8	63.9	2.88

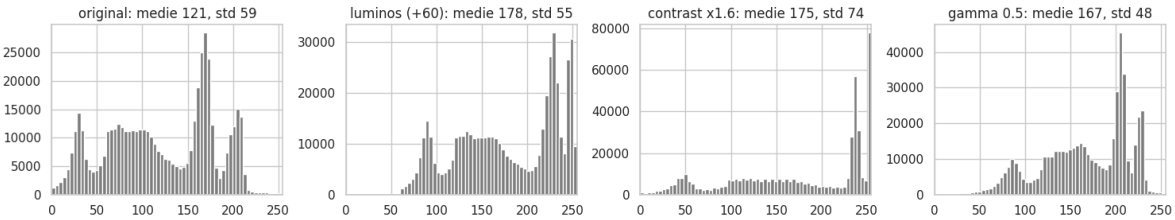
In [16]:
```
show(list(variants.values()), list(variants.keys()), cols=4, size=3.2)
```



In [17]:
```
selected = ['original', 'luminos (+60)', 'contrast x1.6', 'gamma 0.5']
fig, axes = plt.subplots(1, 4, figsize=(15, 3))

for ax, key in zip(axes, selected):
    gray = cv2.cvtColor(variants[key], cv2.COLOR_BGR2GRAY)
    ax.hist(gray.ravel(), bins=64, range=(0, 255), color='gray')
    ax.set_title(f'{key}: medie {gray.mean():.0f}, std {gray.std():.0f}')
    ax.set_xlim(0, 255)

plt.tight_layout()
plt.show()
```



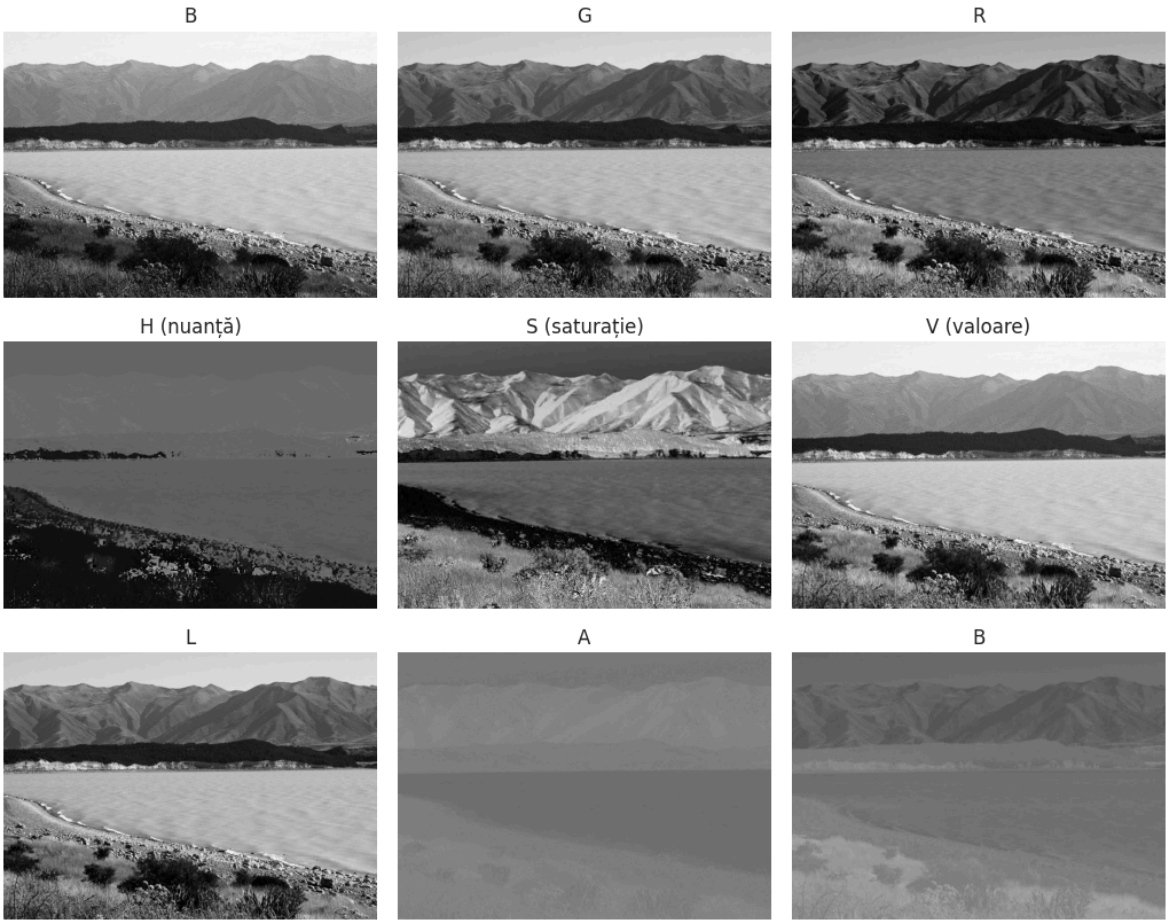
3. Spații de culori

`cv2.cvtColor` convertește între spații de culori; `cv2.split` / `cv2.merge` separă și îmbină canalele. Comparăm canalele imaginii `lac` în **BGR**, **HSV** (nuanță, saturație, valoare) și **LAB** (luminozitate, verde–roșu, albastru–galben).

In [18]:
```
img = IMAGES['lac']
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)

b, g, r = cv2.split(img)
hh, ss, vv = cv2.split(hsv)
L, A, B = cv2.split(lab)

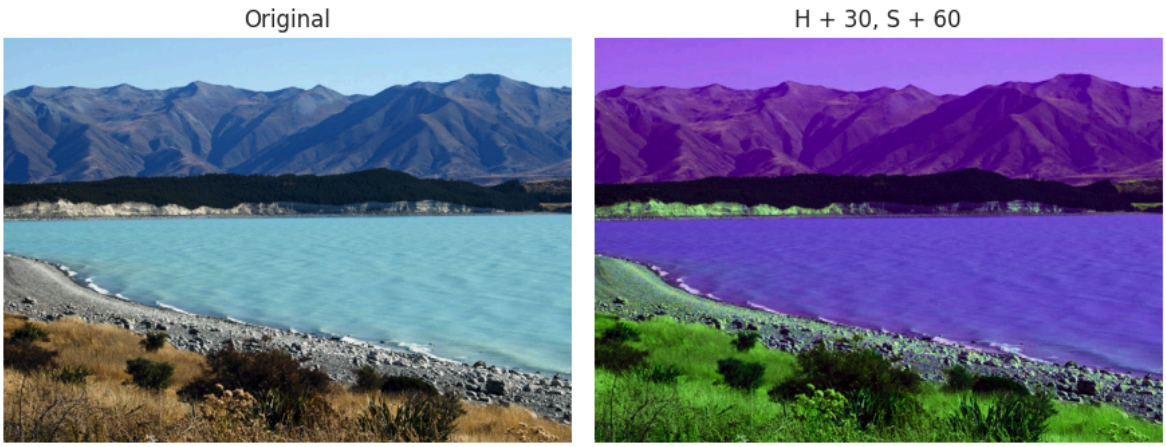
show([b, g, r, hh, ss, vv, L, A, B],
     ['B', 'G', 'R', 'H (nuanță)', 'S (saturație)', 'V (valoare)', 'L', 'A', 'B'],
     cols=3, size=3.5)
```



Modificarea unui singur canal după `split` este un mod simplu de a schimba aspectul imaginii: rotim nuanța (H) cu 60° (în OpenCV `H ∈ [0, 179]`, deci +30) și creștem saturația.

In [19]:
```
h_new = ((hh.astype(np.int16) + 30) % 180).astype(np.uint8)   # rotire ciclică
hsv_mod = cv2.merge((h_new, cv2.add(ss, 60), vv))
img_mod = cv2.cvtColor(hsv_mod, cv2.COLOR_HSV2BGR)

show([img, img_mod], ['Original', 'H + 30, S + 60'], cols=2, size=4.5)
```



Segmentare după culoare: RGB vs HSV

Vrem să segmentăm **apa turcoaz** a lacului. În HSV culoarea este descrisă de un interval de nuanță și saturație, în mare măsură independent de luminozitate, iar `cv2.inRange` produce direct o mască binară. Explorați intervalele interactiv:

In [20]:
```
@interact(h_min=(0, 179, 1), h_max=(0, 179, 1),
          s_min=(0, 255, 5), v_min=(0, 255, 5))
def explore_hsv(h_min=80, h_max=100, s_min=60, v_min=80):
    mask = cv2.inRange(hsv, (h_min, s_min, v_min), (h_max, 255, 255))
    segmented = cv2.bitwise_and(img, img, mask=mask)
    title = f'Mască H∈[{h_min},{h_max}], S≥{s_min}, V≥{v_min}'
    show([mask, segmented], [title, 'Segmentare'], cols=2, size=4.5)
```

Pentru a compara **stabilitatea la iluminare** a celor două reprezentări avem nevoie de o **mască de referință** (rolul unei adnotări manuale): masca HSV de mai sus, curățată prin **operații morfologice** (`cv2.morphologyEx` : *deschiderea* elimină punctele izolate, *închiderea* umple golurile mici) și redusă la componenta cea mai mare. Apoi calibrăm ambele metode pe **aceiași pixeli** de referință, folosind percentilele 2–98 ale valorilor lor:

- **RGB**: un paralelipiped de praguri pe B, G și R;
- **HSV**: praguri pe H și S; pe V doar un prag minim, care exclude pixelii aproape negri (unde nuanța este nedefinită).

In [21]:
```
KERNEL = np.ones((7, 7), np.uint8)

raw_mask = cv2.inRange(hsv, (80, 60, 80), (100, 255, 255))
clean = cv2.morphologyEx(raw_mask, cv2.MORPH_OPEN, KERNEL)
clean = cv2.morphologyEx(clean, cv2.MORPH_CLOSE, KERNEL)
contours, _ = cv2.findContours(clean, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
largest = max(contours, key=cv2.contourArea)

ref_mask = np.zeros_like(raw_mask)
cv2.drawContours(ref_mask, [largest], -1, 255, -1)
print(f'Masca de referință acoperă {100 * ref_mask.mean() / 255:.1f}% din imagine')

px_bgr = img[ref_mask > 0]
px_hsv = hsv[ref_mask > 0]

lo_bgr = np.percentile(px_bgr, 2, axis=0)
hi_bgr = np.percentile(px_bgr, 98, axis=0)
h_lo, h_hi = np.percentile(px_hsv[:, 0], [2, 98])
```

```
s_lo = np.percentile(px_hsv[:, 1], 2)
lo_hsv, hi_hsv = (h_lo, s_lo, 30), (h_hi, 255, 255)
print('Praguri BGR:', lo_bgr.round(), '→', hi_bgr.round())
print('Praguri HSV:', np.round(lo_hsv), '→', np.round(hi_hsv))

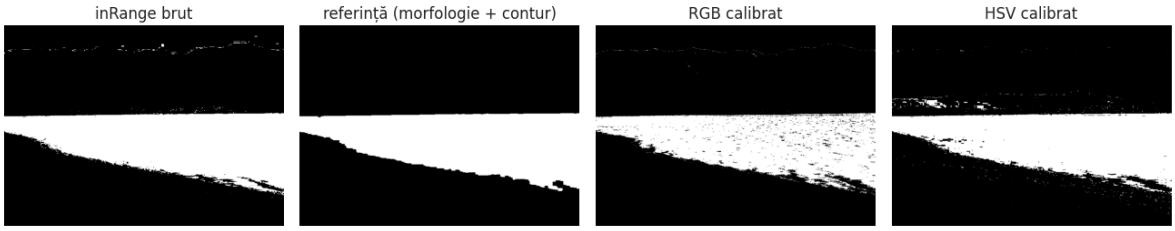

def segment_rgb(bgr):
    return cv2.inRange(bgr, lo_bgr, hi_bgr)


def segment_hsv(bgr):
    return cv2.inRange(cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV), lo_hsv, hi_hsv)


def iou(mask_a, mask_b):
    a, b = mask_a > 0, mask_b > 0
    union = (a | b).sum()
    return (a & b).sum() / union if union else 1.0


show([raw_mask, ref_mask, segment_rgb(img), segment_hsv(img)],
     ['inRange brut', 'referință (morfologie + contur)',
      'RGB calibrat', 'HSV calibrat'], cols=4, size=3.3)
```

```
Masca de referință acoperă 25.2% din imagine
Praguri BGR: [162. 150. 103.] → [212. 197. 141.]
Praguri HSV: [92. 68. 30.] → [ 99. 255. 255.]
```



Simulăm schimbări de iluminare (gamma, scalarea luminozității, supraexpunere, lumină caldă/rece) și măsurăm două lucruri pe pixelii de referință: **cât se schimbă fiecare canal** (diferența medie absolută în BGR și HSV) și **IoU** (Intersection over Union) dintre masca obținută cu pragurile calibrate și masca de referință.

In [22]:
```
def scale_channels(bgr, factors):
    scaled = bgr.astype(np.float32) * np.array(factors, np.float32)
    return np.clip(scaled, 0, 255).astype(np.uint8)


def channel_shift(original, changed, mask):
    o_hsv = cv2.cvtColor(original, cv2.COLOR_BGR2HSV)
    c_hsv = cv2.cvtColor(changed, cv2.COLOR_BGR2HSV)
    d_bgr = np.abs(original[mask > 0].astype(int) - changed[mask > 0].astype(int))
    d_hsv = np.abs(o_hsv[mask > 0].astype(int) - c_hsv[mask > 0].astype(int))
    deltas = np.concatenate([d_bgr.mean(axis=0), d_hsv.mean(axis=0)])
    return dict(zip(['dB', 'dG', 'dR', 'dH', 'dS', 'dV'], deltas.round(1)))


ILLUMINATION = {
    'original':           lambda im: im,
    'gamma 0.6':          lambda im: gamma_correct(im, 0.6),
    'gamma 0.8':          lambda im: gamma_correct(im, 0.8),
    'gamma 1.25':         lambda im: gamma_correct(im, 1.25),
    'gamma 1.6':          lambda im: gamma_correct(im, 1.6),
    'gamma 2.0':          lambda im: gamma_correct(im, 2.0),
    'luminozitate x0.5':  lambda im: scale_channels(im, (0.5, 0.5, 0.5)),
    'luminozitate x0.75': lambda im: scale_channels(im, (0.75, 0.75, 0.75)),
    'luminozitate x1.2':  lambda im: scale_channels(im, (1.2, 1.2, 1.2)),
    'supraexpunere x1.4': lambda im: scale_channels(im, (1.4, 1.4, 1.4)),
    'lumina calda 8%':    lambda im: scale_channels(im, (0.92, 1.0, 1.08)),
```

```
    'lumina rece 8%':     lambda im: scale_channels(im, (1.08, 1.0, 0.92)),
}

rows, masks = [], {}

for label, transform in ILLUMINATION.items():
    changed = transform(img)
    m_rgb, m_hsv = segment_rgb(changed), segment_hsv(changed)
    masks[label] = (changed, m_rgb, m_hsv)

    row = {'iluminare': label, **channel_shift(img, changed, ref_mask)}
    row['IoU_RGB'] = round(iou(m_rgb, ref_mask), 3)
    row['IoU_HSV'] = round(iou(m_hsv, ref_mask), 3)
    rows.append(row)

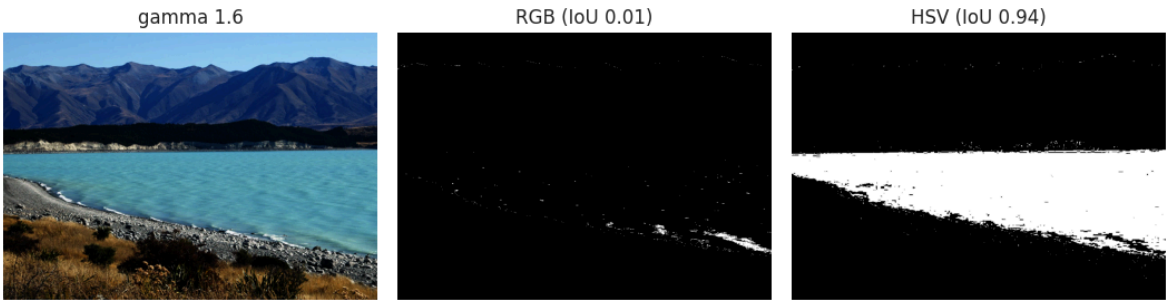
color_df = pd.DataFrame(rows)
save_json(color_df, 'segmentare_culoare.json')
color_df
```

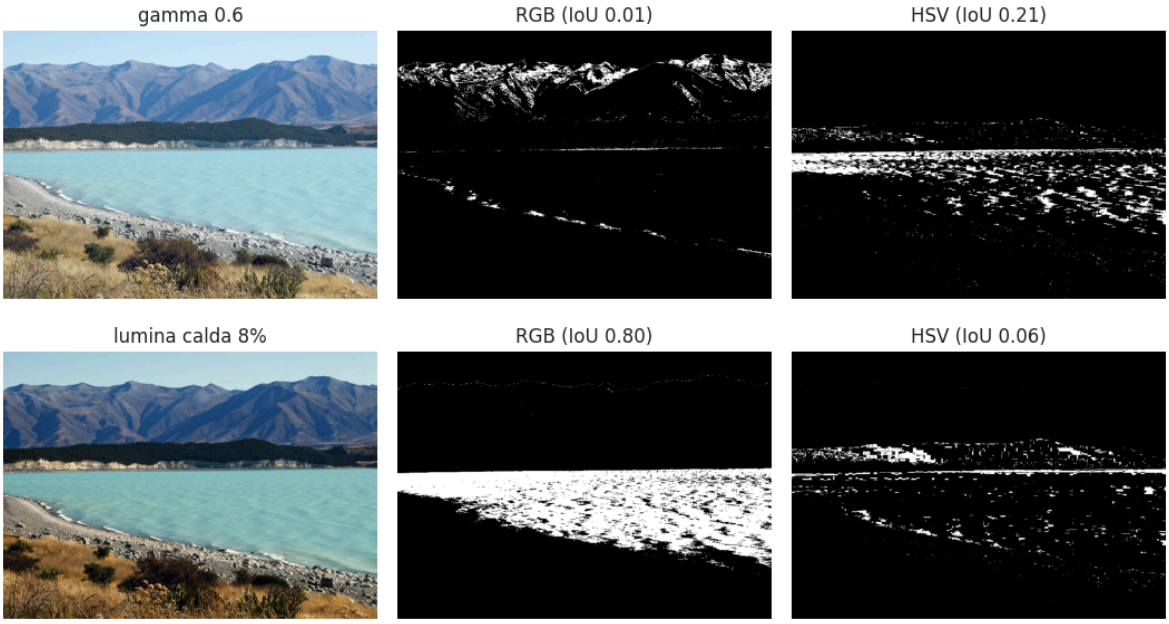
Out[22]:

	iluminare	dB	dG	dR	dH	dS	dV	IoU_RGB	IoU_HSV
0	original	0.0	0.0	0.0	0.0	0.0	0.0	0.922	0.944
1	gamma 0.6	21.3	25.7	41.1	0.5	32.5	21.3	0.007	0.214
2	gamma 0.8	10.0	12.1	18.7	0.3	15.4	10.0	0.409	0.886
3	gamma 1.25	12.9	15.1	20.9	0.2	17.8	12.9	0.473	0.954
4	gamma 1.6	28.8	33.6	43.9	0.6	39.2	28.8	0.009	0.943
5	gamma 2.0	45.3	52.0	63.8	1.0	59.7	45.3	0.000	0.929
6	luminozitate x0.5	98.2	91.3	61.7	0.3	0.7	98.2	0.000	0.958
7	luminozitate x0.75	49.4	45.9	31.1	0.2	0.5	49.4	0.001	0.951
8	luminozitate x1.2	38.8	36.0	24.2	0.1	0.3	38.8	0.039	0.904
9	supraexpunere x1.4	57.4	67.3	48.7	4.3	12.8	57.4	0.006	0.281
10	lumina calda 8%	16.1	0.0	9.4	7.2	24.5	13.2	0.801	0.061
11	lumina rece 8%	15.2	0.0	10.3	3.2	24.0	15.2	0.374	0.816

In [23]:

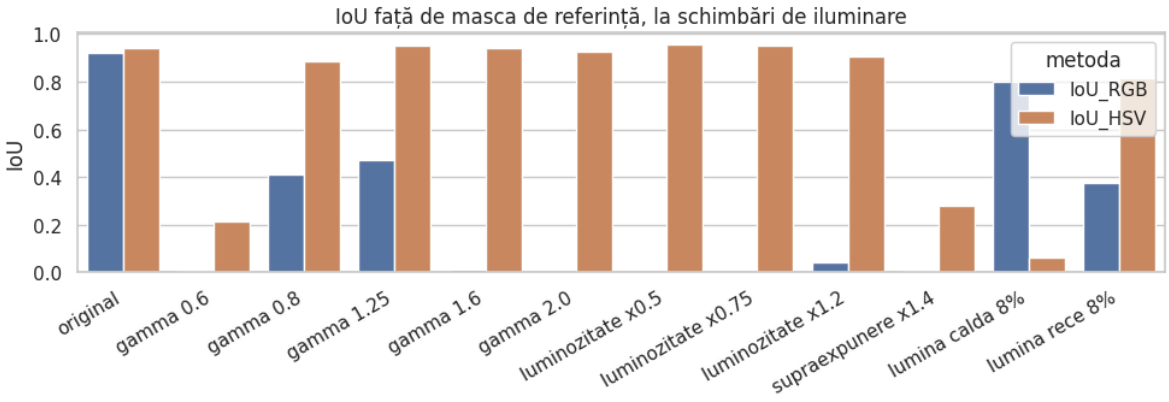
```
for label in ['gamma 1.6', 'gamma 0.6', 'lumina calda 8%']:
    changed, m_rgb, m_hsv = masks[label]
    titles = [label, f'RGB (IoU {iou(m_rgb, ref_mask):.2f})',
              f'HSV (IoU {iou(m_hsv, ref_mask):.2f})']
    show([changed, m_rgb, m_hsv], titles, cols=3, size=3.6)
```





In [24]:
```
long_df = color_df.melt(id_vars='iluminare', value_vars=['IoU_RGB', 'IoU_HSV'],
                        var_name='metoda', value_name='IoU')

plt.figure(figsize=(10, 3.6))
sns.barplot(data=long_df, x='iluminare', y='IoU', hue='metoda')
plt.xticks(rotation=30, ha='right')
plt.xlabel('')
plt.title('IoU față de masca de referință, la schimbări de iluminare')
plt.tight_layout()
plt.show()
```



4. Praguri și măști

Binarizarea unui document fotografiat este primul pas în OCR. Comparăm pe imaginea `pagina` (iluminare puternic neuniformă) pragul **global** (`cv2.threshold`), pragul **Otsu** (ales automat din histogramă) și pragul **adaptiv** (`cv2.adaptiveThreshold`, calculat local, dintr-o vecinătate a fiecărui pixel).

In [25]:
```
def global_threshold(im, t=127):
    return cv2.threshold(im, t, 255, cv2.THRESH_BINARY)[1]


def otsu_threshold(im):
    return cv2.threshold(im, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)[1]
```

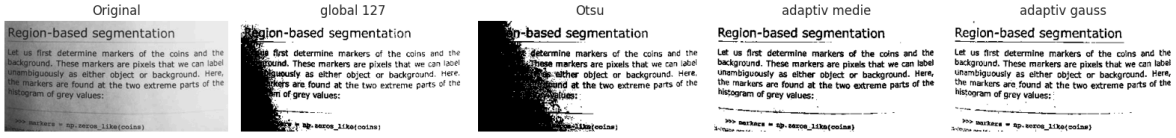
```
def adaptive_threshold(im, method, block=31, c=12):
    return cv2.adaptiveThreshold(im, 255, method, cv2.THRESH_BINARY, block, c)


MEAN, GAUSS = cv2.ADAPTIVE_THRESH_MEAN_C, cv2.ADAPTIVE_THRESH_GAUSSIAN_C
METHODS = {
    'global 127':    global_threshold,
    'Otsu':          otsu_threshold,
    'adaptiv medie': lambda im: adaptive_threshold(im, MEAN),
    'adaptiv gauss': lambda im: adaptive_threshold(im, GAUSS),
}

page = IMAGES['pagina']
otsu_t, _ = cv2.threshold(page, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print('Prag Otsu:', otsu_t)

show([page] + [m(page) for m in METHODS.values()],
     ['Original'] + list(METHODS), cols=5, size=3.2)
```

```
Prag Otsu: 157.0
```



Vizual, pragul adaptiv câștigă, dar pentru o evaluare **numerică** avem nevoie de adevărul de bază (pixelii care sunt text). Îl obținem cu un **document sintetic**: randăm text cu `cv2.putText` (masca de text este cunoscută exact), apoi îl degradăm cu un gradient de iluminare de intensitate controlată și zgomot gaussian. Zgomotul se reduce cu un filtru median 3×3 înainte de binarizare (același pas pentru toate metodele; filtrarea se studiază detaliat în lecția 5). Măsurăm scorul **F1** al pixelilor de text pentru fiecare metodă și fiecare intensitate a gradientului.

In [26]:
```
LINES = ['Vederea computerizata', 'OpenCV + NumPy', 'Praguri: global, Otsu, adaptiv',
         'Lucrare de laborator 1', 'F1 = 2PR / (P + R)']


def synthetic_document(size=(360, 640), text_val=60, paper_val=230):
    doc = np.full(size, paper_val, np.uint8)
    for i, text in enumerate(LINES):
        cv2.putText(doc, text, (25, 60 + i * 65), cv2.FONT_HERSHEY_SIMPLEX,
                    1.1, text_val, 2, cv2.LINE_AA)
    return cv2.GaussianBlur(doc, (3, 3), 0)     # margini ușor neclare, ca la o poză


def degrade(doc, gradient, noise=8, seed=0):
    h, w = doc.shape
    ramp = np.linspace(1.0, 1.0 - gradient, w, dtype=np.float32)  # scade spre dreapta
    rng = np.random.default_rng(seed)
    out = doc.astype(np.float32) * ramp[None, :] + rng.normal(0, noise, doc.shape)
    return np.clip(out, 0, 255).astype(np.uint8)


def f1_score(pred_text, true_text):
    tp = np.sum(pred_text & true_text)
    fp = np.sum(pred_text & ~true_text)
    fn = np.sum(~pred_text & true_text)
    return 2 * tp / (2 * tp + fp + fn)


doc = synthetic_document()
true_text = doc < 145
rows = []
```

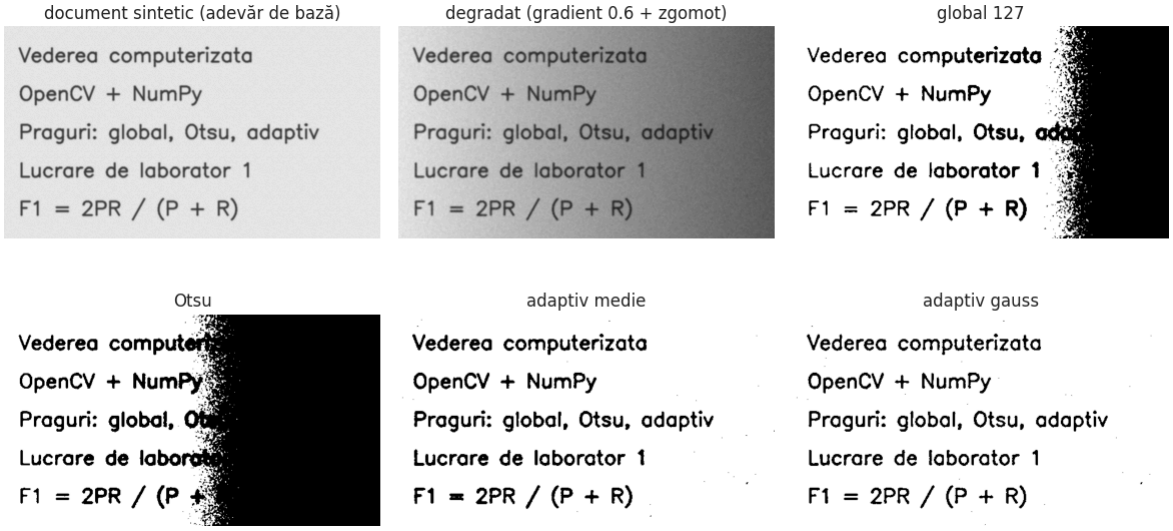
```
for gradient in [0.0, 0.2, 0.4, 0.6, 0.8]:
    degraded = cv2.medianBlur(degrade(doc, gradient), 3)
    for name, method in METHODS.items():
        f1 = f1_score(method(degraded) < 128, true_text)
        rows.append({'gradient': gradient, 'metoda': name, 'F1': round(f1, 3)})

thresh_df = pd.DataFrame(rows)
save_json(thresh_df, 'praguri.json')
thresh_df.pivot(index='gradient', columns='metoda', values='F1')[list(METHODS)]
```

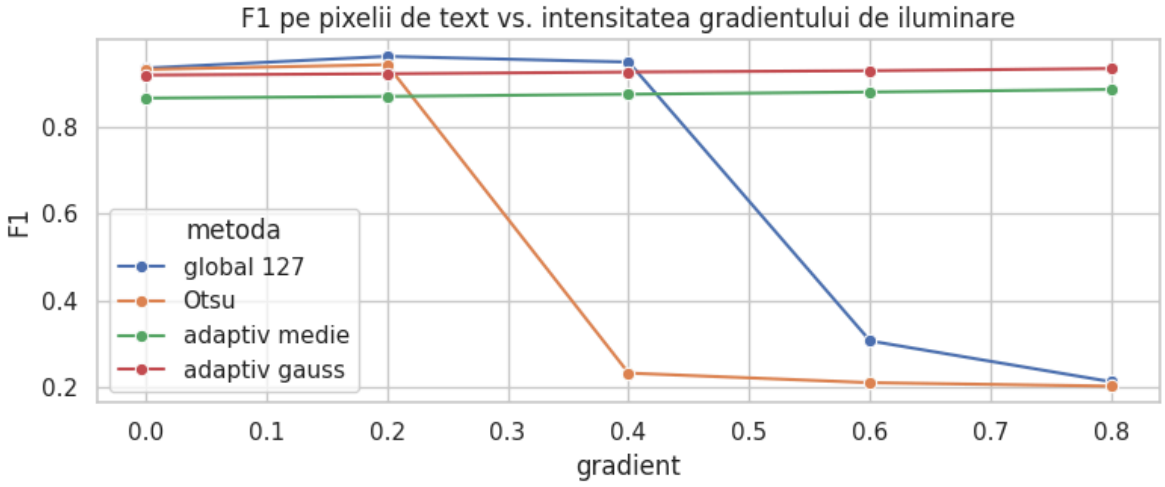
Out[26]:

gradient	global 127	Otsu	adaptiv medie	adaptiv gauss
0.0	0.934	0.931	0.865	0.918
0.2	0.961	0.942	0.869	0.921
0.4	0.948	0.233	0.874	0.925
0.6	0.307	0.211	0.879	0.928
0.8	0.213	0.203	0.885	0.933

In [27]:
```
degraded = cv2.medianBlur(degrade(doc, 0.6), 3)
show([doc, degraded] + [m(degraded) for m in METHODS.values()],
     ['document sintetic (adevăr de bază)', 'degradat (gradient 0.6 + zgomot)']
     + list(METHODS), cols=3, size=4)
```



In [28]:
```
plt.figure(figsize=(8, 3.5))
sns.lineplot(data=thresh_df, x='gradient', y='F1', hue='metoda', marker='o')
plt.title('F1 pe pixelii de text vs. intensitatea gradientului de iluminare')
plt.tight_layout()
plt.show()
```



Măști și operații pe biți

O mască binară selectează pixelii pe care îi modificăm. Suprapunem literele logo-ului peste colțul lacului: pragul pe imaginea gri a logo-ului dă masca literelor; `cv2.bitwise_and` cu masca inversată „golește” fundalul, iar `cv2.add` combină prim-planul cu fundalul. Comparativ, `cv2.addWeighted` amestecă două imagini cu ponderi (blending), fără mască.

In [29]:
```
img = IMAGES['lac']
logo = cv2.resize(IMAGES['logo'], (300, 300), interpolation=cv2.INTER_AREA)

logo_gray = cv2.cvtColor(logo, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(logo_gray, 200, 255, cv2.THRESH_BINARY)   # literele albe
mask_inv = cv2.bitwise_not(mask)

lh, lw = mask.shape
roi = img[-lh - 20:-20, -lw - 20:-20]                    # colțul dreapta-jos
background = cv2.bitwise_and(roi, roi, mask=mask_inv)    # fundal fără litere
foreground = cv2.bitwise_and(logo, logo, mask=mask)      # doar literele

watermarked = img.copy()
watermarked[-lh - 20:-20, -lw - 20:-20] = cv2.add(background, foreground)

astro = cv2.resize(IMAGES['astronaut'], (img.shape[1], img.shape[0]))
blended = cv2.addWeighted(img, 0.65, astro, 0.35, 0)

show([mask, mask_inv, background, foreground],
     ['mască', 'mască inversată', 'fundal (ROI ∧ ¬mască)',
      'prim-plan (logo ∧ mască)'], cols=4, size=3)
show([watermarked, blended], ['Watermark prin măști', 'addWeighted 0.65 / 0.35'],
     cols=2, size=5)
```



Watermark prin măști



addWeighted 0.65 / 0.35



5. Adnotare și colaj

Adnotăm rezultatul segmentării: din masca de referință a apei extragem conturul (`cv2.findContours`), desenăm dreptunghiul de încadrare (`cv2.boundingRect`), centrul de masă (`cv2.moments`) și o etichetă cu `cv2.putText`. Adăugăm și o linie de orizont. Coordonatele sunt `(x, y)`, iar culorile sunt în ordinea **BGR**.

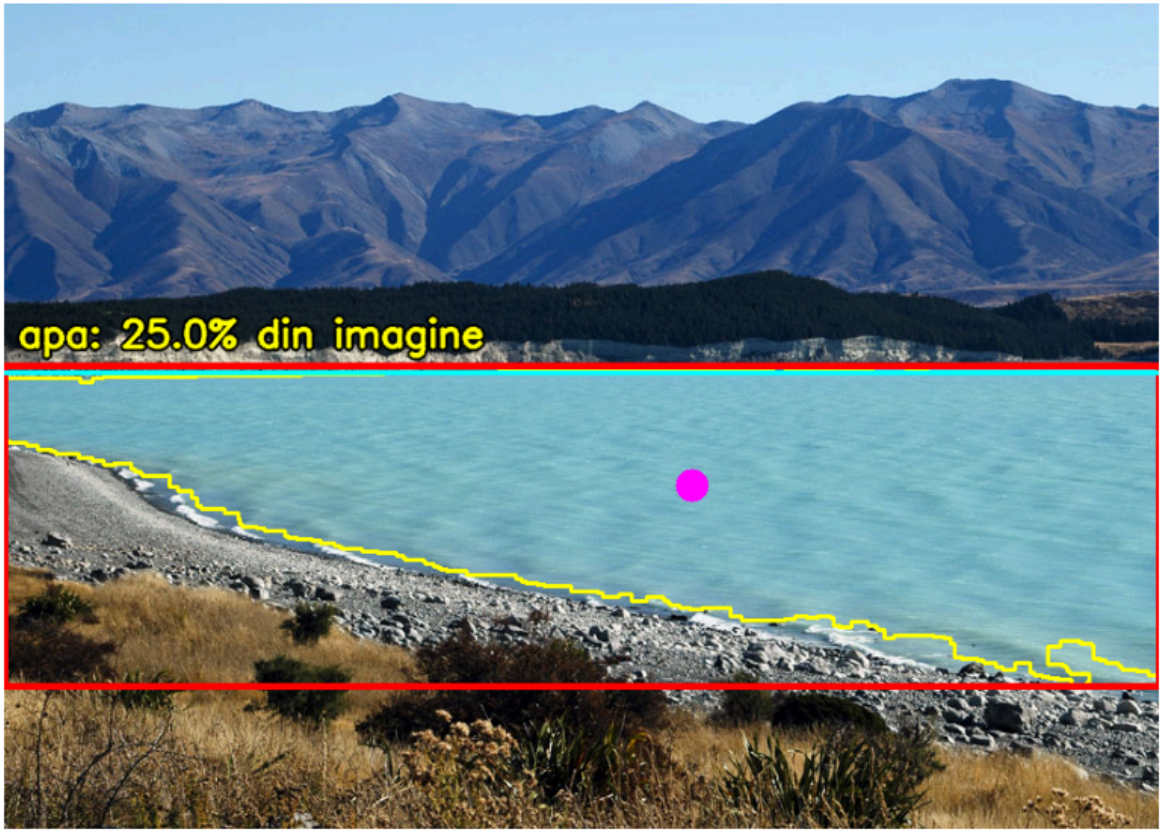
In [30]:
```
annotated = IMAGES['lac'].copy()
contours, _ = cv2.findContours(ref_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
largest = max(contours, key=cv2.contourArea)

x, y, w_box, h_box = cv2.boundingRect(largest)
m = cv2.moments(largest)
cx, cy = int(m['m10'] / m['m00']), int(m['m01'] / m['m00'])
area_pct = 100 * cv2.contourArea(largest) / (annotated.shape[0] * annotated.shape[1])

cv2.drawContours(annotated, [largest], -1, (0, 255, 255), 2)
cv2.rectangle(annotated, (x, y), (x + w_box, y + h_box), (0, 0, 255), 3)
cv2.circle(annotated, (cx, cy), 12, (255, 0, 255), -1)
cv2.line(annotated, (0, 268), (annotated.shape[1], 268), (255, 255, 0), 2,
         cv2.LINE_AA)
label = f'apa: {area_pct:.1f}% din imagine'
for color, thick in [((0, 0, 0), 6), ((0, 255, 255), 2)]:      # contur + text
    cv2.putText(annotated, label, (x + 10, y - 14), cv2.FONT_HERSHEY_SIMPLEX,
                0.9, color, thick, cv2.LINE_AA)

show([annotated], ['Segmentare adnotată'], size=8)
```

Segmentare adnotată



Colajul combină rezultatele de mai sus într-o singură imagine: fiecare piesă este redimensionată la aceeași dimensiune (imaginile gri sunt convertite la 3 canale), primește o etichetă și este așezată în grilă cu `np.hstack` / `np.vstack`.

In [31]:
```python
def make_collage(tiles, cols=3, tile_size=(320, 230)):
    canvases = []

    for image, label in tiles:
        tile = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR) if image.ndim == 2 else image
        tile = cv2.resize(tile, tile_size, interpolation=cv2.INTER_AREA)
        cv2.rectangle(tile, (0, 0), (tile_size[0], 30), (0, 0, 0), -1)
        cv2.putText(tile, label, (8, 21), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                    (255, 255, 255), 1, cv2.LINE_AA)
        canvases.append(tile)

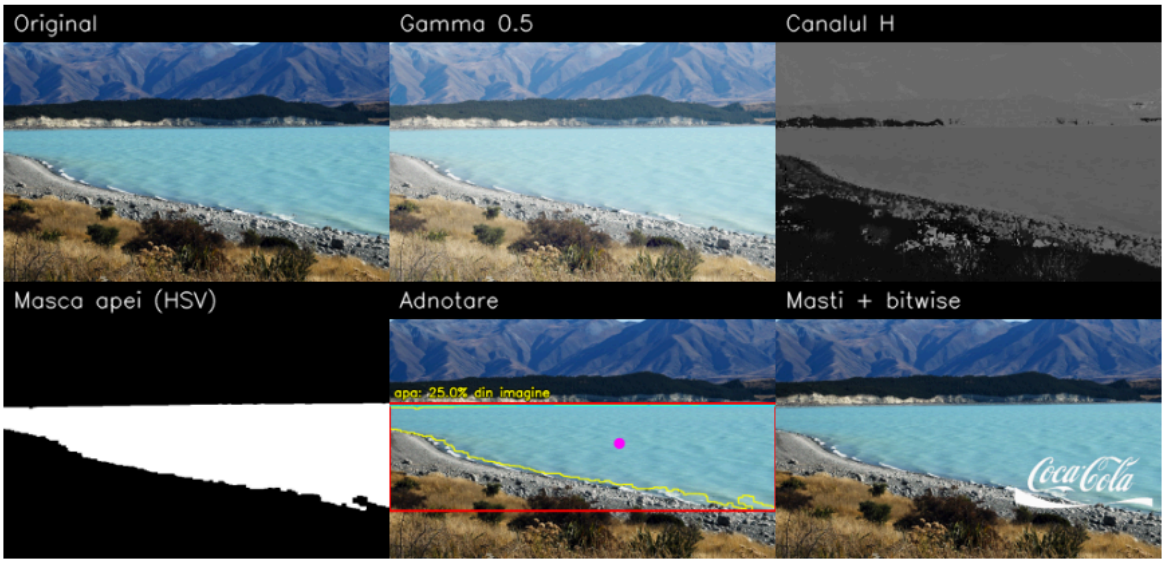
    while len(canvases) % cols:
        canvases.append(np.zeros_like(canvases[0]))

    rows = [np.hstack(canvases[i:i + cols]) for i in range(0, len(canvases), cols)]
    return np.vstack(rows)


collage = make_collage([
    (IMAGES['lac'], 'Original'),
    (variants['gamma 0.5'], 'Gamma 0.5'),
    (hh, 'Canalul H'),
    (ref_mask, 'Masca apei (HSV)'),
    (annotated, 'Adnotare'),
    (watermarked, 'Masti + bitwise'),
])

cv2.imwrite(os.path.join(OUT_DIR, 'colaj.jpg'), collage)
show([collage], ['Colaj (rezultate/colaj.jpg)'], size=9)
```

Colaj (rezultate/colaj.jpg)



6. Video: pipeline cadru cu cadru

Un videoclip este o secvență de imagini, deci tot ce am făcut mai sus se aplică pe fiecare cadru citit cu `cv2.VideoCapture`. Clipul `roboti.mp4` (terminal portuar, noaptea) conține containere albastre pe care le detectăm după culoare:

1. redimensionare la 50% (viteză);
2. conversie BGR → HSV și `cv2.inRange` pentru albastru;
3. curățarea măștii prin morfologie (deschidere + închidere);
4. contururi → dreptunghiuri de încadrare (obiectele cu arie mică sunt ignorate) → adnotare;
5. scrierea cadrului cu `cv2.VideoWriter`.

Măsurăm timpul de procesare al fiecărui cadru pentru a stabili dacă pipeline-ul poate rula în timp real (≥ FPS-ul clipului).

In [32]:
```
BLUE_LO, BLUE_HI = (95, 80, 60), (130, 255, 255)
KERNEL5 = np.ones((5, 5), np.uint8)
MIN_AREA = 150


def process_frame(frame):
    small = cv2.resize(frame, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
    mask = cv2.inRange(cv2.cvtColor(small, cv2.COLOR_BGR2HSV), BLUE_LO, BLUE_HI)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, KERNEL5)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, KERNEL5)

    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    boxes = [cv2.boundingRect(c) for c in contours
             if cv2.contourArea(c) >= MIN_AREA]

    for x, y, w, h in boxes:
        cv2.rectangle(small, (x, y), (x + w, y + h), (0, 255, 255), 2)

    return small, mask, len(boxes)
```

In [33]:
```
cap = cv2.VideoCapture(VIDEO_PATH)
assert cap.isOpened(), f'Nu pot deschide {VIDEO_PATH}'
```

```
fps_in = cap.get(cv2.CAP_PROP_FPS)
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
print(f'{VIDEO_PATH}: {width}x{height}, {fps_in:.0f} fps, '
      f'{n_frames} cadre ({n_frames / fps_in:.1f} s)')

out_path = os.path.join(OUT_DIR, 'roboti_adnotat.mp4')
writer = cv2.VideoWriter(out_path, cv2.VideoWriter_fourcc(*'mp4v'), fps_in,
                         (width // 2, height // 2))

timing, samples = [], {}
idx = 0

while True:
    ok, frame = cap.read()
    if not ok:
        break

    start = time.perf_counter()
    result, mask, n_obj = process_frame(frame)
    ms = (time.perf_counter() - start) * 1000

    cv2.putText(result, f'cadru {idx}  obiecte: {n_obj}  {ms:.1f} ms', (10, 25),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2, cv2.LINE_AA)
    writer.write(result)

    timing.append({'cadru': idx, 'timp_ms': ms, 'obiecte': n_obj})
    if idx % 100 == 0:
        samples[idx] = (result, mask)
    idx += 1

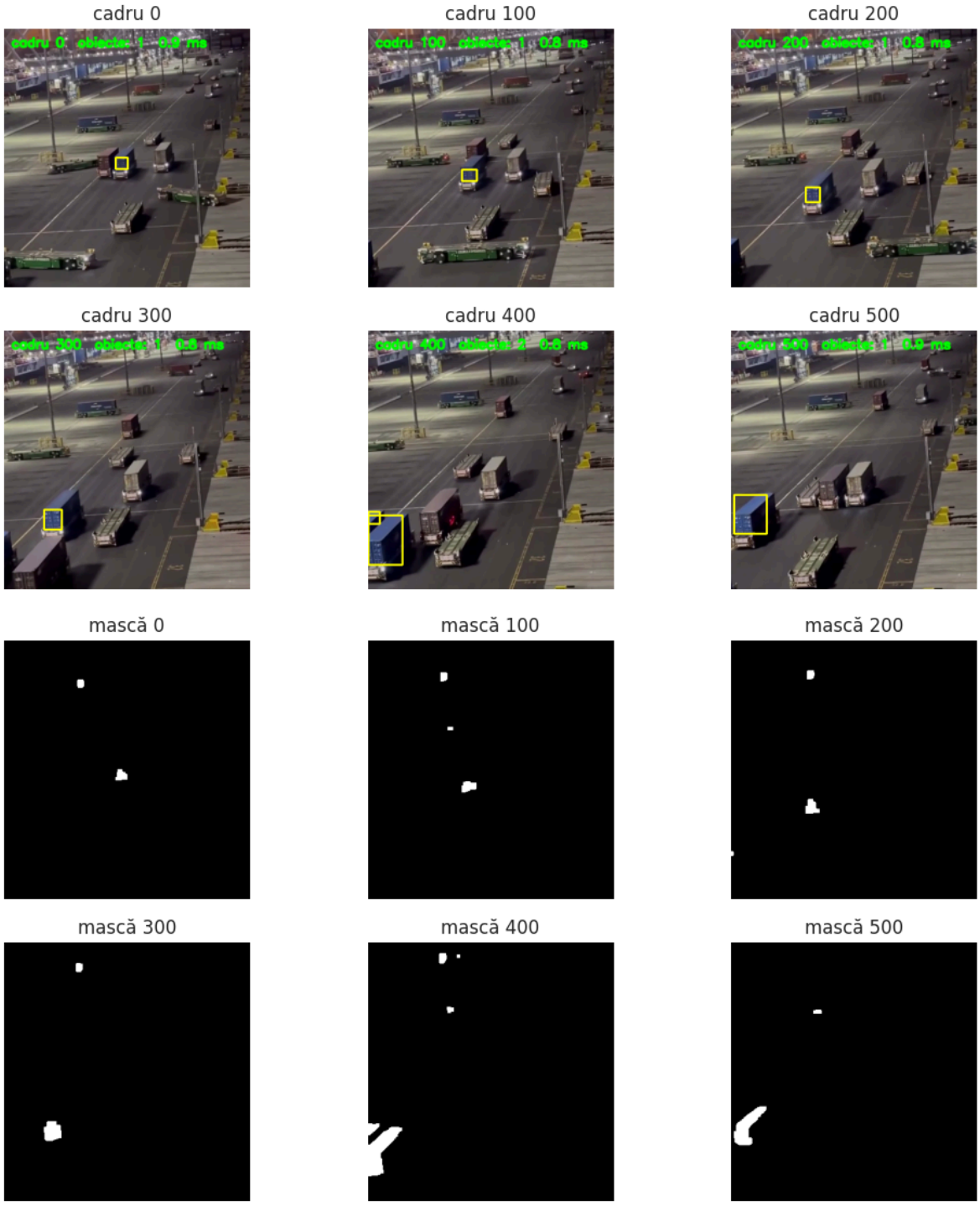
cap.release()
writer.release()

video_df = pd.DataFrame(timing)
save_json(video_df, 'video_timing.json')
print(f'Salvat: {out_path} ({idx} cadre)')
```

```
roboti.mp4: 694x732, 30 fps, 585 cadre (19.5 s)
Salvat: rezultate/roboti_adnotat.mp4 (585 cadre)
```

In [34]:

```
show([r for r, _ in samples.values()], [f'cadru {i}' for i in samples],
     cols=3, size=3.6)
show([m for _, m in samples.values()], [f'mască {i}' for i in samples],
     cols=3, size=3.6)
```



In [35]:
```
video_stats = pd.Series({
    'cadre': len(video_df),
    'timp_mediu_ms': video_df.timp_ms.mean(),
    'timp_median_ms': video_df.timp_ms.median(),
    'timp_p95_ms': video_df.timp_ms.quantile(0.95),
    'fps_procesare': 1000 / video_df.timp_ms.mean(),
    'fps_clip': fps_in,
    'obiecte_medii_per_cadru': video_df.obiecte.mean(),
}).round(2)

video_stats
```

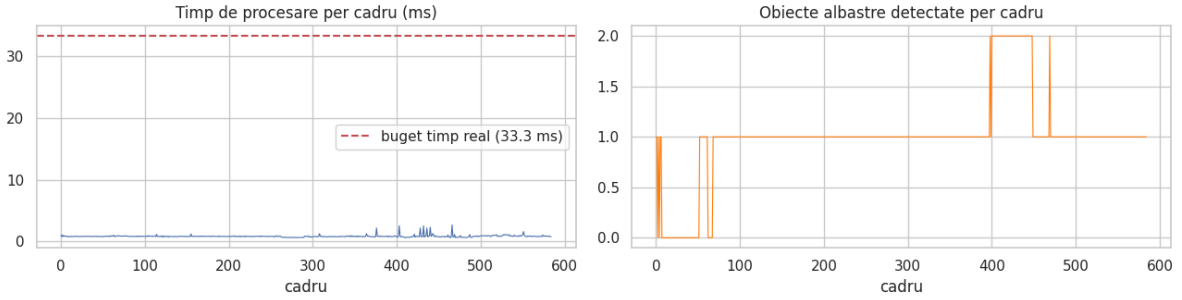
Out[35]:
```
cadre                       585.00
timp_mediu_ms                 0.85
timp_median_ms                0.82
timp_p95_ms                   1.03
fps_procesare              1180.09
fps_clip                     30.00
obiecte_medii_per_cadru       1.00
dtype: float64
```

In [36]:
```
budget_ms = 1000 / fps_in
fig, axes = plt.subplots(1, 2, figsize=(13, 3.5))

axes[0].plot(video_df.cadru, video_df.timp_ms, lw=0.8)
axes[0].axhline(budget_ms, color='r', ls='--',
                label=f'buget timp real ({budget_ms:.1f} ms)')
axes[0].set_title('Timp de procesare per cadru (ms)')
axes[0].set_xlabel('cadru')
axes[0].legend()

axes[1].plot(video_df.cadru, video_df.obiecte, lw=0.8, color='tab:orange')
axes[1].set_title('Obiecte albastre detectate per cadru')
axes[1].set_xlabel('cadru')

plt.tight_layout()
plt.show()
```



Agregarea rezultatelor

Toate experimentele au fost salvate în `rezultate/*.json`. Le reîncărcăm și extragem indicatorii principali într-un singur tabel, ca bază pentru concluzii.

In [37]:
```
results = {Path(f).stem: pd.read_json(f)
           for f in sorted(glob(os.path.join(OUT_DIR, '*.json')))}
print({k: len(v) for k, v in results.items()})

interp = results['interpolare'].groupby(['rol', 'metoda']).mean(numeric_only=True)
shrink_res, enlarge_res = interp.loc['micsorare'], interp.loc['marire']
color = results['segmentare_culoare'].set_index('iluminare')
praguri = results['praguri'].groupby('metoda').F1.agg(['mean', 'min'])

summary = pd.Series({
    'micșorare: cel mai bun PSNR / SSIM':
        f'{shrink_res.psnr.idxmax()} / {shrink_res.ssim.idxmax()}',
    'mărire: cel mai bun PSNR / SSIM':
        f'{enlarge_res.psnr.idxmax()} / {enlarge_res.ssim.idxmax()}',
    'mărire: cea mai rapidă / cea mai lentă':
        f'{enlarge_res.timp_ms.idxmin()} / {enlarge_res.timp_ms.idxmax()}',
    'segmentare culoare: IoU mediu RGB / HSV':
        f'{color.IoU_RGB.mean():.3f} / {color.IoU_HSV.mean():.3f}',
    'segmentare culoare: IoU minim RGB / HSV':
        f'{color.IoU_RGB.min():.3f} / {color.IoU_HSV.min():.3f}',
    'praguri: F1 mediu (cea mai bună metodă)':
        f"{praguri['mean'].idxmax()} ({praguri['mean'].max():.3f})",
    'praguri: F1 minim (cea mai slabă metodă)':
```

```
        f"{praguri['min'].idxmin()} ({praguri['min'].min():.3f})",
    'video: fps procesare / fps clip':
        f'{video_stats.fps_procesare:.0f} / {fps_in:.0f}',
})

summary.to_frame('valoare')
```

```
{'interpolare': 50, 'luminozitate_contrast': 8, 'praguri': 20, 'segmentare_culoare': 12,
'video_timing': 585}
```

Out[37]:

	valoare
micșorare: cel mai bun PSNR / SSIM	AREA / AREA
mărire: cel mai bun PSNR / SSIM	LANCZOS4 / LANCZOS4
mărire: cea mai rapidă / cea mai lentă	NEAREST / LANCZOS4
segmentare culoare: IoU mediu RGB / HSV	0.253 / 0.737
segmentare culoare: IoU minim RGB / HSV	0.000 / 0.061
praguri: F1 mediu (cea mai bună metodă)	adaptiv gauss (0.925)
praguri: F1 minim (cea mai slabă metodă)	Otsu (0.203)
video: fps procesare / fps clip	1180 / 30

Concluzii

Interpolare: metoda potrivită depinde de direcție

- La **micșorare**, `INTER_AREA` dă cea mai bună calitate (PSNR ≈ 26.5 dB, SSIM ≈ 0.79), pentru că mediază toți pixelii acoperiți; `INTER_NEAREST` este cea mai rapidă, dar produce aliasing (PSNR ≈ 22.8 dB, SSIM ≈ 0.68).
- La **mărire**, `INTER_LANCZOS4` și `INTER_CUBIC` sunt practic la egalitate (PSNR ≈ 26.4–26.5 dB, SSIM ≈ 0.79), dar `LANCZOS4` este de ≈ 6× mai lent; `INTER_AREA` la mărire se comportă ca `INTER_NEAREST`.
- Compromisul recomandat: `AREA` pentru micșorare, `CUBIC` pentru mărire.

Luminozitate și contrast: saturarea este pierdere de informație

- Adunarea cu NumPy produce wrap-around (culori false); `cv2.add` / `cv2.convertScaleAbs` saturează corect, echivalent cu `np.clip`.
- Creșterea liniară a contrastului (×1.6) saturează ≈ 33% din pixeli, iar +60 la luminozitate ≈ 16%; corecția **gamma 0.5** luminează imaginea (media 123 → 168) fără pixeli saturați suplimentari (0.28%), fiind varianta sigură pentru scene întunecate.

Spații de culori: nuanța este stabilă la intensitate, nu și la culoarea luminii

- Pe pixelii apei, gamma și scalarea luminozității modifică B, G, R cu 10–98 de unități, dar nuanța **H cu sub 1 unitate**; saturația S rămâne constantă la scalarea luminozității (dS < 1), însă se schimbă la gamma (dS 15–60).

- Pragurile **RGB** calibrate eșuează la orice schimbare de intensitate (IoU ≤ 0.47, de regulă ≈ 0) și rezistă doar la lumina caldă (0.80), unde canalele se modifică puțin; pragurile **HSV** păstrează IoU 0.89–0.96 pentru gamma 0.8–2.0 și luminozitate ×0.5–×1.2.
- HSV eșuează în două situații: (a) imagine puternic luminată (gamma 0.6, supraexpunere ×1.4) — saturația scade sub pragul calibrat, iar clipping-ul la 255 distorsionează nuanța; (b) schimbarea **culorii** luminii (lumină caldă) — nuanța apei se deplasează cu ≈ 7 unități și iese din fereastra calibrată. Pentru (b) este nevoie de corecția balansului de alb, nu de alt spațiu de culori.

Praguri: la iluminare neuniformă, pragul adaptiv

- Fără gradient sau cu gradient mic (≤ 0.2), pragul global și Otsu sunt cele mai bune (F1 ≈ 0.93–0.96), pentru că histograma este clar bimodală.
- De la gradient 0.4 Otsu se prăbușește (F1 ≈ 0.23), iar pragul global de la 0.6 (F1 ≈ 0.31); pragul **adaptiv gaussian** rămâne constant (F1 ≈ 0.92–0.93) indiferent de gradient, iar cel cu medie este puțin mai slab (≈ 0.87–0.89).

Video: pipeline-ul rulează în timp real

- La rezoluție înjumătățită, un cadru se procesează în ≈ 0.9 ms, adică peste 1000 de cadre/s — de ≈ 40× mai rapid decât bugetul de 33 ms al unui clip de 30 fps; detecția pe culoare găsește în medie un container albastru per cadru.
- Rezerva de timp lasă loc pentru operații mai scumpe (filtrare, urmărire, rețele neuronale) în laboratoarele următoare.